Ocean Modelling Collaborative Project: Code Porting and Writing

Solving the Ekman Equation

Rebecca, Nqubeko, Ndumiso and Rob

Introduction to the Ekman Equation

- Describes the **change** in ocean current speed and direction with depth
- Driven by **wind stress** at the ocean surface (McPhaden et al., 2024).
- Wind transfers momentum from the atmosphere to the ocean
- The **Coriolis effect** causes current deflection (Roach et al., 2015).
- Current speed decreases with depth
- Current direction rotates with increasing depth
- Produces a spiral pattern of current velocity

Historical Background

- **Fridtjof Nansen** observed the phenomenon
- **Vagn Walfrid Ekman** developed the scientific theory (Dritschel et al., 2020)
- Ekman's work provided the theoretical basis for the **Ekman spiral** and **Ekman transport**
- Research was encouraged by **Vilhelm Bjerknes**

Process

Wind stress + Coriolis effect → Ekman spiral → Ekman transport → Upwelling/Downwelling → Influences nutrient distribution

Important Ekman Concepts (SEA2004F)

Ekman Theory: wind driven transport in the surface layer of the ocean is directed perpendicular to the wind (left in the SH)

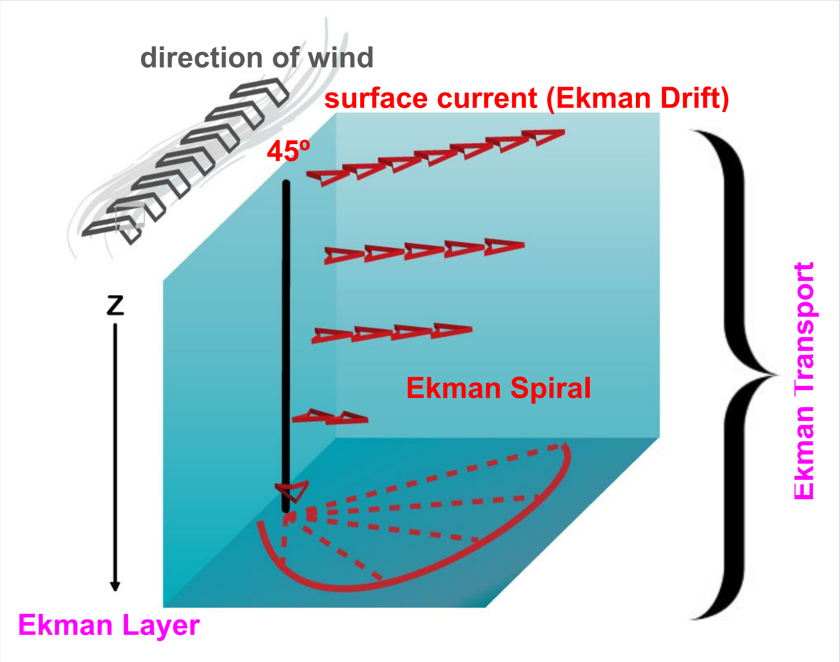
Ekman Drift: wind-driven movement of the surface layer of the ocean

Ekman Layer: depth at which current velocity is opposite to surface velocity

Ekman Transport: transport of water due to Ekman Drift

Ekman Spiral

- wind → kinetic energy + friction → surface layer
- kinetic energy propagates to lower layers through friction → decreases in each layer
- spiral of moving water (100 - 150 m → this is the Ekman Layer)
- average direction of all moving water is 90º from the wind direction → this is the Ekman Transport



The Ekman Equation: Discretized Forms of Zonal and Meridional Components (Exercise 13)

explicit Euler method used to discretize the equations → iterate through time → approximate solution to **Ekman Spiral**

Zonal

$$U_k^{n+1} = U_k^n + \Delta t \left[f V_{E\,k}^n + A_v \frac{U_{k-1}^n - 2U_k^n + U_{k+1}^n}{\Delta z^2} \right]$$

Meridional

$$V_k^{n+1} = V_k^n + \Delta t \left[-f U_k^n + A_v \frac{V_{k-1}^n - 2V_k^n + V_{k+1}^n}{\Delta z^2} \right]$$

Stability Constraint
(minimum time step used)

$$\Delta t < \frac{\Delta z^2}{2 A_v} = \frac{\Delta z^2}{2 \times 10^{-4}}$$

Python Code Part 1 - 3: Setting up the Model

```
%matplotlib inline

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

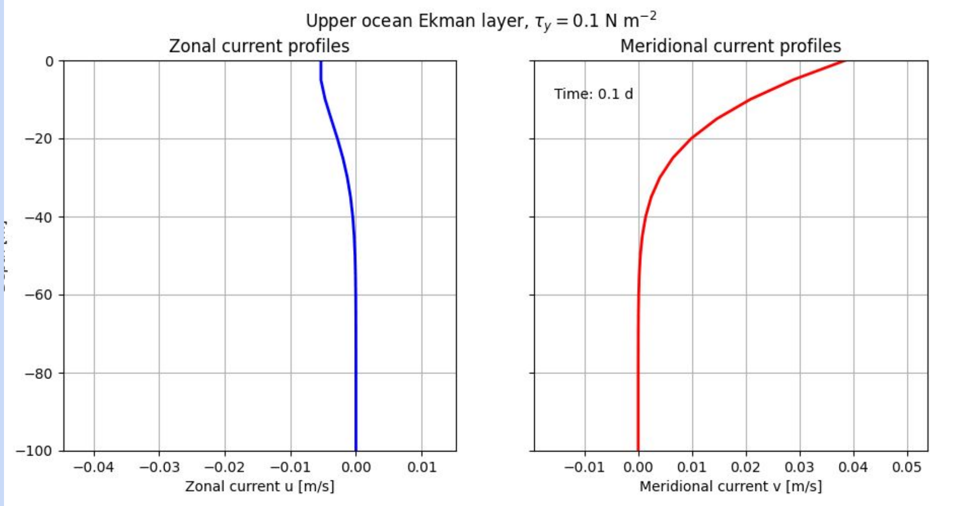
- Inputs: grid depth & resolution, wind stress, eddy viscosity (A), latitude
- Latitude → Coriolis parameter (f) and inertial period
- Calculate CFL condition ($w = A \cdot dt/dz^2 < 0.5$) for a numerically stable solution
- Initial condition: ocean starts at rest ($u = v = 0$ everywhere)

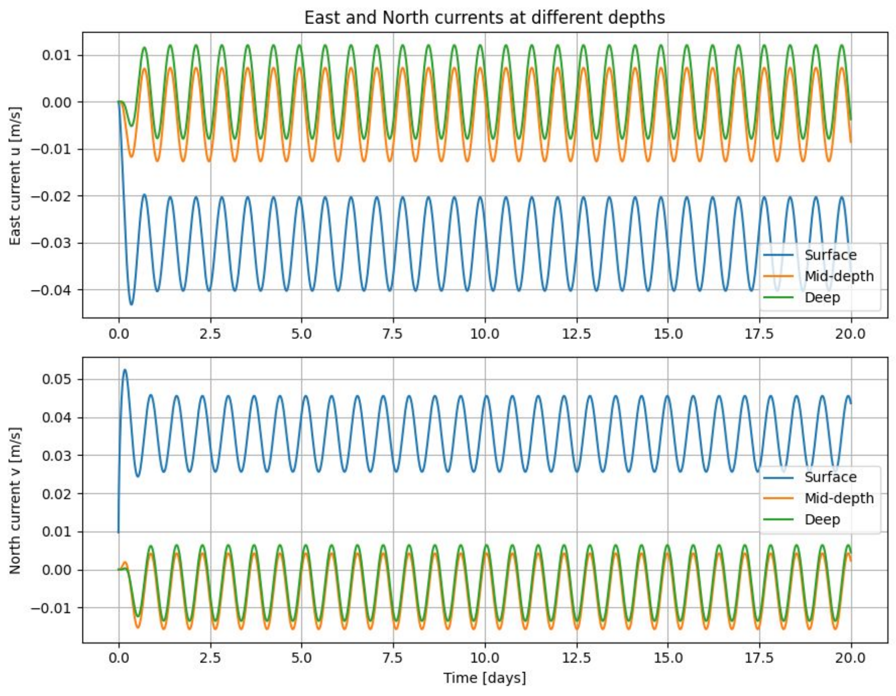
Python Code Part 4: Time Stepping Loop

```python
for n in range(nstep):

    # advance time
    time[n] = n * dt

    # evaluate the diffusion term
    delsqu = np.zeros_like(u)
    delsqu[1:-1] = u[:-2] - 2.0*u[1:-1] + u[2:]

    delsqv = np.zeros_like(v)
    delsqv[1:-1] = v[:-2] - 2.0*v[1:-1] + v[2:]

    # Euler forward solution with Coriolis
    u = u + w*delsqu + dt*f*v
    v = v + w*delsqv - dt*f*u

    # surface boundary condition: wind stress forcing
    u[0] = u[1] + dz*(Tauwx/rho0/A)
    v[0] = v[1] + dz*(Tauwy/rho0/A)

    # bottom boundary condition: Neumann / free-slip
    u[-1] = u[-2]
    v[-1] = v[-2]
```

```python
    # time-average after the first day
    if time[n] / SECPERDAY >= 1.0:
        uavg = uavg + u
        vavg = vavg + v
        navg = navg + 1

    # time series at selected depths
    usurf[n] = u[0]
    vsurf[n] = v[0]

    # Converted to Python indexing:
    mid_idx = int(np.round(2*nz/3)) - 2

    umid[n] = u[mid_idx]
    vmid[n] = v[mid_idx]

    udeep[n] = u[-4]
    vdeep[n] = v[-4]

    # transport
    transu[n] = (np.sum(u) - u[0]) * dz
    transv[n] = (np.sum(v) - v[0]) * dz
```

Python Code Part 5 and 6:

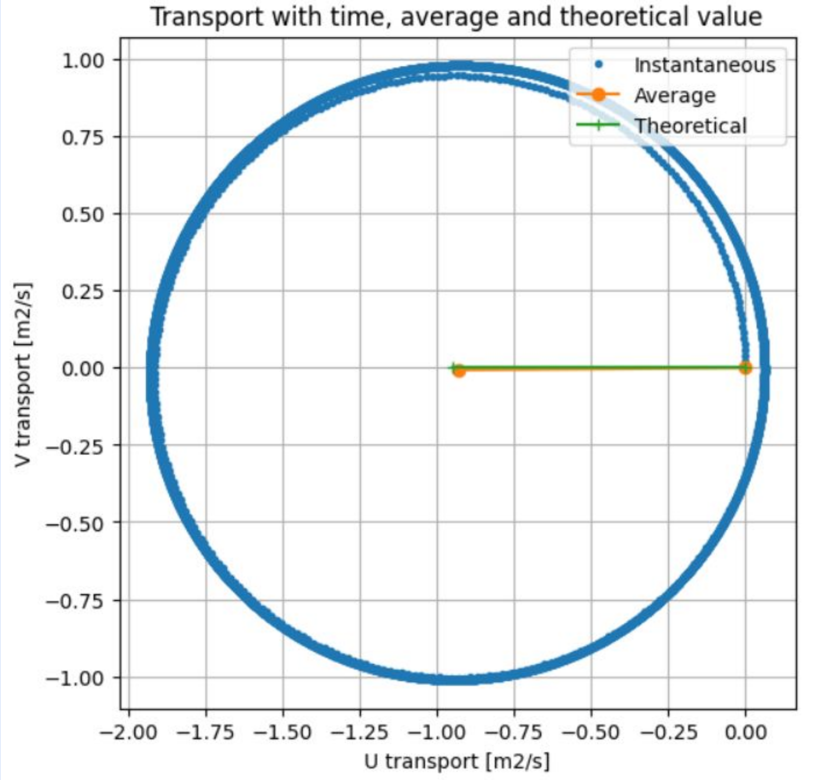




- Large latitude → dominated by reaction term → steeper vertical gradients
- Small latitude → dominated by diffusion → flatter profile

Python Code Part 7: Ekman transport

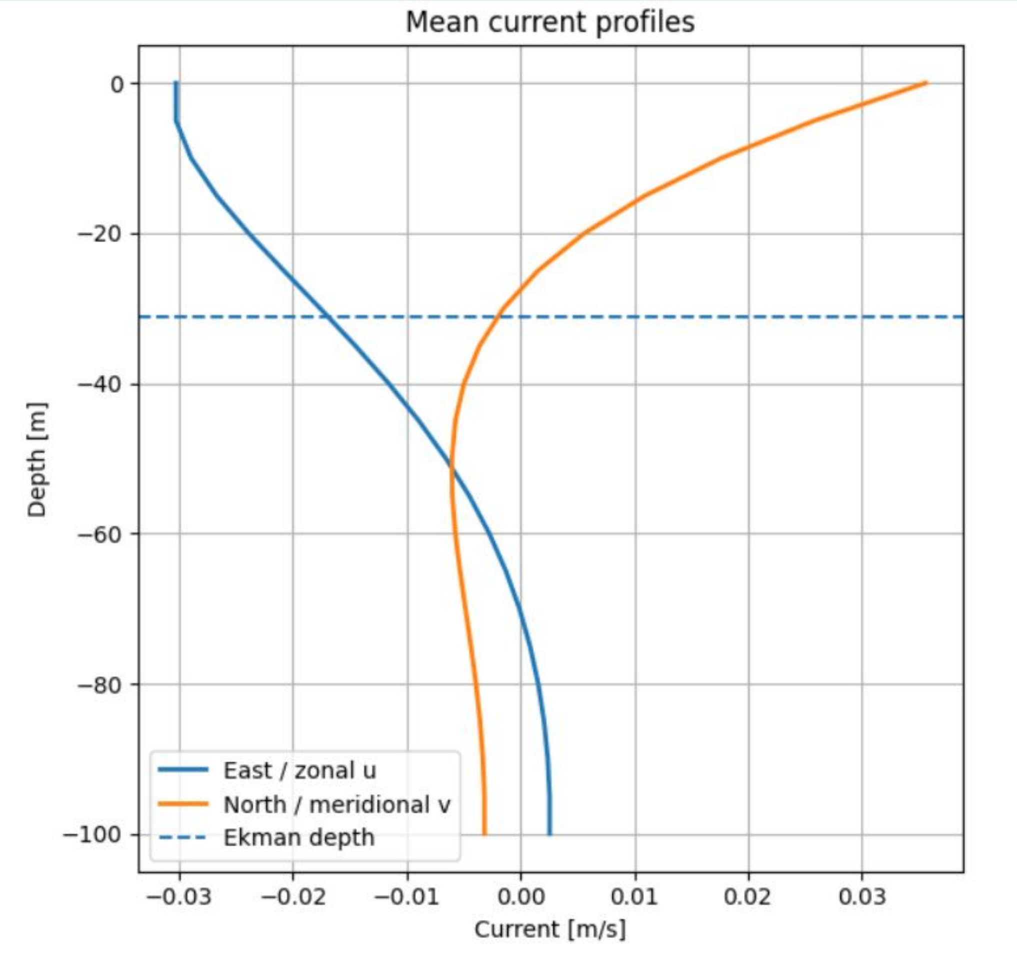
```python
# theoretical Ekman transport and Ekman depth
Ektrans = np.sqrt(Tauwx**2 + Tauwy**2) / (rho0 * f)
Ekdepth = np.sqrt(2*A / np.abs(f))

Tx = np.mean(transu)
Ty = np.mean(transv)

plt.figure(figsize=(6, 6))
plt.plot(transu, transv, ".", label="Instantaneous")
plt.plot([0, Tx], [0, Ty], "o-", label="Average")
plt.plot([0, Ektrans], [0, 0], "+-", label="Theoretical")

plt.xlabel("U transport [m2/s]")
plt.ylabel("V transport [m2/s]")
plt.title("Transport with time, average and theoretical value")
plt.axis("equal")
plt.grid(True)
plt.legend()
plt.show()

print("Mean U transport:", Tx, "m2/s")
print("Mean V transport:", Ty, "m2/s")
print("Theoretical Ekman transport:", Ektrans, "m2/s")
print("Ekman depth:", Ekdepth, "m")
```




```
Mean U transport: -0.9292336825294137 m2/s
Mean V transport: -0.007086744504506565 m2/s
Theoretical Ekman transport: -0.9463103900251566 m2/s
Ekman depth: 31.144311676063502 m
```

Python Code Part 8: Mean Velocity Profile

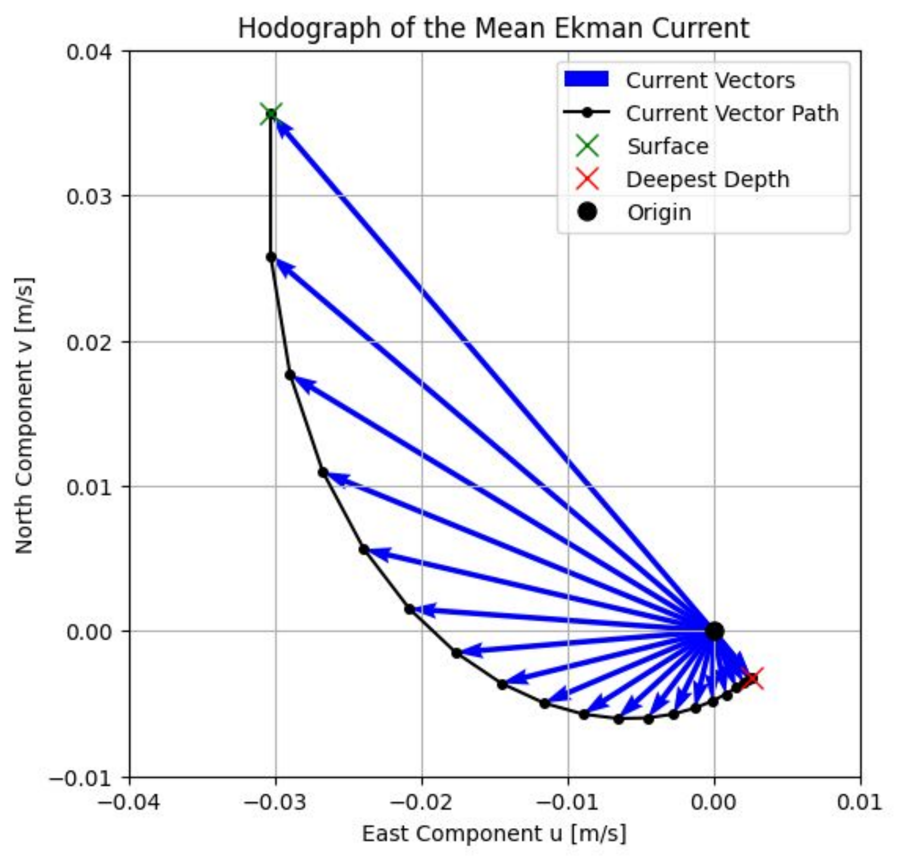
```
plt.figure(figsize=(6, 6))

plt.plot(uavg, z, label="East / zonal u", linewidth=2)
plt.plot(vavg, z, label="North / meridional v", linewidth=2)

plt.axhline(-Ekdepth, linestyle="--", label="Ekman depth")

plt.xlabel("Current [m/s]")
plt.ylabel("Depth [m]")
plt.title("Mean current profiles")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```



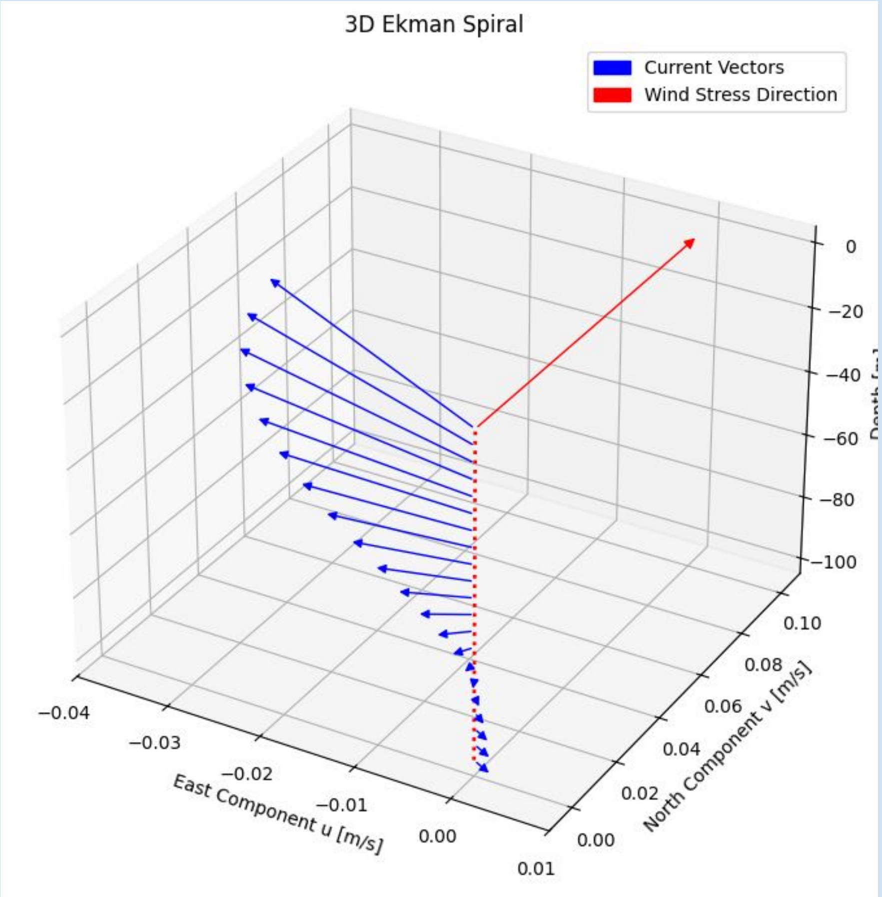
Python Code Part 9: Hodograph

shows the change in strength and direction of the current velocity vectors with depth

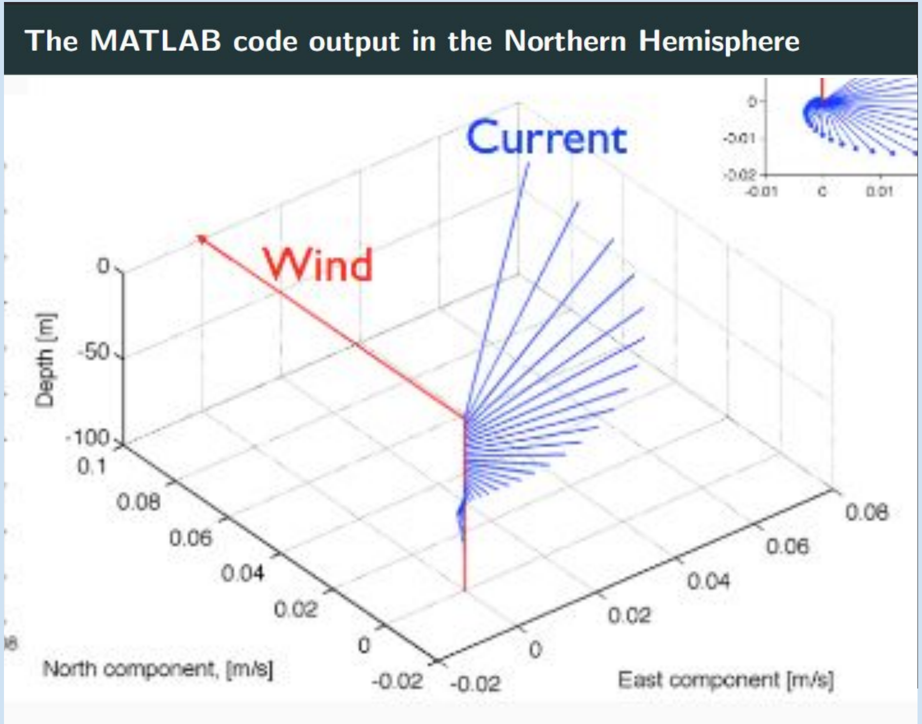
```
zeroz = np.zeros_like(uavg)# np.zeroz is a numpy array similar to “uavg” (same data
shape and type) but all elements are initialised to zero → starting point

plt.figure(figsize=(6, 6))
plt.quiver(zeroz, zeroz, uavg, vavg, angles="xy", scale_units="xy", scale=1,
color='blue', label='Current Vectors')
# arrows with “zeroz” as origin and “uavg/vavg” as end-point coordinates (length and
direction of arrows)

# add a line connecting the tips of the arrows to show the path
plt.plot(uavg, vavg,'k-o', markersize=4, label='Current Vector Path')
# add a marker at the shallowest (surface) and deepest points and at the origin
plt.plot(uavg[0], vavg[0], 'gx', markersize=10, label='Surface')
plt.plot(uavg[-1], vavg[-1], 'rx', markersize=10, label='Deepest Depth')
plt.plot(0, 0, 'ko', markersize=8, label='Origin')

# set axis limits manually
plt.xlim(-0.04, 0.01)
plt.ylim(-0.01, 0.04)

# add labels, titles and a legend
plt.xlabel("East Component u [m/s]")
plt.ylabel("North Component v [m/s]")
plt.title("Hodograph of the Mean Ekman Current")
plt.grid(True)
plt.legend()
plt.show()
```

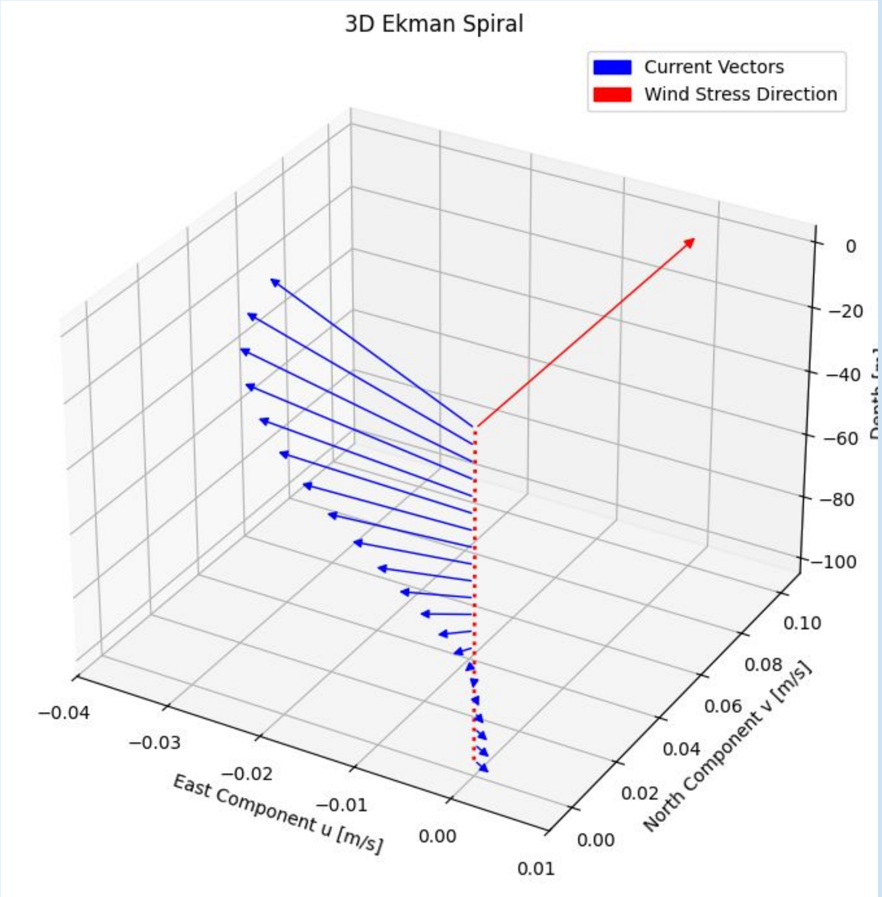


```python
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection="3d")
zeroz = np.zeros_like(uavg)

# plot current vectors using Arrow3D: loop creates an arrow at each depth
for i in range(len(z)):
 lbl = "Current Vectors" if i == 0 else "" # label on 1st iteration for legend
 arrow = Arrow3D([0, uavg[i]], [0, vavg[i]], [z[i], z[i]], mutation_scale=10,
arrowstyle="-|>", color="blue", label=lbl)
 ax.add_artist(arrow)
 # start and end coordinates for each arrow are indicated
 # "mutation scale" controls size of arrow head and length

# vertical red reference line ("-L" is max. depth)
ax.plot([0, 0], [0, 0], [-L, 0], linewidth=2, color="red", linestyle=":")

# plot wind stress direction vector also an an Arrow3D object
wind_arrow = Arrow3D([0, 0], [0, 0.1], [0, 0], mutation_scale=12, arrowstyle="-|>",
color="red", label="Wind Stress Direction")
ax.add_artist(wind_arrow)

# set axis limits to reduce empty space
ax.set_xlim(-0.04, 0.01)
ax.set_ylim(-0.01, 0.11)
ax.set_zlim(-105, 5)

# add labels, titles and a legend
ax.set_xlabel("East Component u [m/s]", labelpad=10)
ax.set_ylabel("North Component v [m/s]", labelpad=10)
ax.set_zlabel("Depth [m]")
ax.set_title("3D Ekman Spiral")
ax.legend()
plt.show()
```



lecture slides show that the 3D profile is for the NH → in the code: f and latitude are negative which indicates code for the SH → the 3D plots are the same though → possible mistake in lecture slides





Advantages and Disadvantages of the Model

✅ **CONCEPTUAL CLARITY**

- clearly shows the balance between wind stress, Coriolis and friction

✅ **PREDICTS EKMAN TRANSPORT**

- explains and calculates the net movement of water perpendicular to the wind.

✅ **SHOWS DEPTH DIFFERENCE**

- demonstrates how current speed decreases and direction changes with depth = Ekman Spiral

✅ **SIMPLE AND EFFICIENT**

- easy to understand, computationally inexpensive, and useful for education and initial investigations.

❌ **IDEALISED ASSUMPTIONS**

- homogenous ocean and ignores stratification
- constant eddy viscosity (A) so ignores turbulence and mixing
- constant and uniform wind stress
- only forces considered are Coriolis and wind
- simplified boundary conditions (depth)

❌ **LACKS THE ABILITY TO PREDICT REAL WORLD CONDITIONS AND SCENARIOS**

❌ **CANNOT CAPTURE SMALL-SCALE PROCESSES**

- parameterised complex turbulent processes through eddy viscosity rather than resolving them

Group Roles

Software Designers and Programmers: Rob and Ndumiso

Researchers (Background Theory and Concepts): Nqubeko and Rebecca

Advantages and Disadvantages: Nqubeko and Rebecca

Code/Plot Improvement on Collab Notebook: all

References

Dritschel, D. G., Paldor, N., and Constantin, A. 2020. The Ekman spiral for piecewise-uniform viscosity, Ocean Science., 16:1089–1093. https://doi.org/10.5194/os-16-1089-2020.

McPhaden, M. J., Athulya, K., Girishkumar, M. S., & Orlić, M. (2024). Ekman revisited: Surface currents to the left of the winds in the Northern Hemisphere. *Science advances*, *10*(46), eadr0282. https://doi.org/10.1126/sciadv.adr0282

Roach, C. J., Phillips, H. E., Bindoff, N. L., & Rintoul, S. R. (2015). Detecting and Characterizing Ekman Currents in the Southern Ocean. *Journal of Physical Oceanography*, *45*(5), 1205-1223. https://doi.org/10.1175/JPO-D-14-0115.1